# TLS · Istio Ambient · Cilium



# 01 TLS에 대해서



# TLS = Transport Layer Security

**Transport** — 일반적으로 **TCP 연결 위에서 동작**한다. TCP가 만들어 놓은 신뢰성 있는 바이트 스트림을 이용한다.

**Layer** — 애플리케이션과 TCP 사이에서 **암호화·인증·무결성 보호를 제공하는 논리적인 계층**이라는 의미다. OSI 모델의 특정 계층을 뜻하는 것은 아니다. HTTP뿐 아니라 SMTP, IMAP 등 TCP 기반 프로토콜에서도 사용할 수 있다.

**Security** — 통신 상대 인증, 데이터 암호화, 데이터 무결성을 제공한다.

이전 이름은 SSL(Secure Sockets Layer)이었다. 이후 표준화 과정에서 TLS(Transport Layer Security)로 이어졌으며, 지금도 관행적으로 TLS 인증서를 "SSL 인증서"라고 부르는 경우가 많다.

# TLS가 보장하는 세 가지

| 무엇 | 어떤 원리로 |
| --- | --- |
| 기밀성 | 데이터를 열쇠로 암호화해서 보낸다 |
| 무결성 | 보낼 때 검사값을 같이 붙이고, 받는 쪽이 다시 계산해 맞춰본다 |
| 인증 | 상대가 낸 인증서와, 그 인증서의 개인키로 만든 서명을 검증한다 |

# 공개키와 대칭키 — 어디에 쓰나

| 구간 | 무엇을 쓰나 | 왜 |
|---|---|---|
| 핸드셰이크 — 키 합의 | **공개키(비대칭)** | 미리 정한 비밀 없이도 처음 만난 상대와 공유 비밀을 만들 수 있다 |
| 핸드셰이크 — 상대 확인 | **공개키(서명·인증서)** | 개인키를 가진 쪽만 서명을 만들 수 있다 |
| 실제 데이터 전송 | **대칭키** | 비대칭은 훨씬 느리다. 양이 많은 본 통신에는 못 쓴다 |

TLS는 둘 중 하나를 고르는 게 아니라 **역할을 나눠 둘 다 쓴다**.

# 버전은 세 개 — 1.1 · 1.2 · 1.3

| 버전 | 한 줄 |
|---|---|
| 1.1 (2006) | 지원하는 암호화 수준이 낮다 → **쓰지 말 것** |
| 1.2 (2008) | 암호화 수준 문제는 해결. 다만 **통신 횟수가 많다** |
| 1.3 (2018) | 통신 횟수를 줄였다. **지금 기준** |

지금 쓰이는 건 **1.2와 1.3** 두 개다. 1.2는 오래된 상대와 붙어야 할 때 남아 있다.

## 1.1 — 암호화 수준이 낮아서 금지

쓸 수 있는 해시가 **MD5와 SHA-1**뿐이었다. 둘 다 지금 기준으로는 약하다.

암호화와 위조검사를 **따로 처리**했고, 둘을 붙이는 순서에서 공격이 계속 나왔다.

더 센 알고리즘으로 바꾸자고 **협상할 필드가 프로토콜에 없었다.** 그래서 프로토콜을 갈아치우는 수밖에 없었다.

1.0도 함께 금지

# 1.2 — 문제는 고쳤지만 통신이 많다

1. 클라이언트: 인사 + 내가 쓸 수 있는 방식 목록
2. 서버: 이 방식으로 하자 + 내 인증서
3. 클라이언트: 그 방식에 맞춰 키 정보 전송
4. 서버: 확인 완료 — 이제야 데이터

열쇠를 맞추는 데만 **왕복이 두 번** 필요하다. 방식을 정한 다음에 키 정보를 보내는 순서였기 때문이다.

# 1.3 — 통신을 어떻게 줄였나

**1.2 — 왕복 두 번**

- 방식을 정한 다음에 키 정보를 보낸다
- 서버 확인까지 끝나야 데이터가 나간다

**1.3 — 왕복 한 번**

- 첫 인사에 키 교환 그룹과 공개값을 미리 넣는다
- 서버가 그 그룹을 받아들이면 그 자리에서 키 완성
- 서버가 다른 그룹을 원하면 되물어본다 → 왕복 두 번

클라이언트가 **어느 그룹으로 키를 만들지 미리 찍어서** 공개값까지 같이 보낸다. 맞으면 한 번, 틀리면 서버가 되물어서 두 번이다.

**별개 변화** — 1.3은 암호 조합 이름에서 키 교환과 인증을 떼어내 따로 협상하게 바꿨다. 그래서 등록된 조합이 300개 이상에서 5개로 줄었다. 이건 이름과 협상의 단순화이고, 위의 왕복 감소와는 다른 이야기다.

# 02 TLS 통신 방법

ClientHello · ServerHello · 그리고 mTLS

# ClientHello — 실제로 나가는 내용

```
ClientHello                              # 전체가 평문
  supported_versions  TLS 1.3, TLS 1.2       # 쓸 수 있는 버전
  cipher_suites       TLS_AES_128_GCM_SHA256 # 암호 방식 후보
  key_share           x25519 + 공개값 32B    # 키 만들 그룹 + 내 공개값
  server_name         api.example.com        # 접속할 도메인
  alpn                h2, http/1.1           # 그 위에서 쓸 프로토콜
```

`server_name`과 `alpn`이 **평문으로 그대로 보인다**는 걸 여기서 확인할 수 있다.

# ServerHello — 서버가 고른 결과

```
ServerHello
  supported_versions  TLS 1.3                # 고른 버전
  cipher_suite        TLS_AES_128_GCM_SHA256  # 후보 중 하나만
  key_share           x25519 + 공개값 32B     # 서버 쪽 공개값
──────────────── 여기까지 평문 ────────────────
  {EncryptedExtensions} {Certificate} {Finished}
```

서버 공개값이 오면 양쪽 열쇠가 완성된다. 그래서 **그 다음 줄부터 암호화된다.**

# TLS → mTLS: 무엇이 추가되나

### TLS — 서버만 인증

- 서버만 Certificate 제시
- 클라이언트는 검증만 한다
- 서버는 상대가 누구인지 모른다
- 앱 레벨 토큰·API 키로 따로 보완

### mTLS — 상호 인증

- 서버가 CertificateRequest 를 추가
- 클라이언트도 Certificate + CertificateVerify
- 양쪽이 서로를 암호학적으로 확인
- 대신 인증서 배포·갱신 부담이 생긴다

# 순서로 보면 두 줄이 늘어난다

**TLS**

- → ClientHello + 내 공개값
- ← ServerHello + 서버 공개값
- ← {서버 인증서 + 서명 + Finished}
- → {Finished} — 데이터 시작

**mTLS**

- → ClientHello + 내 공개값
- ← ServerHello + 서버 공개값
- ← {서버 인증서 + 서명 + 인증서 요청 + Finished}
- → {클라이언트 인증서 + 서명 + Finished}

늘어난 것은 **서버의 인증서 요청**과 **클라이언트의 인증서·서명** 둘뿐이다. 왕복 수는 그대로다.

# 03 Istio

Service Mesh 구현체 · k8s Add-On · istiod



# Service Mesh — 앱에서 무엇을 빼내나

**앱이 직접 하던 것**

- mTLS 라이브러리를 코드에 링크
- 리트라이 · 타임아웃 · 서킷브레이커 구현
- 언어마다 다시 구현 (Java, Go, Python…)
- 정책 하나 바꾸려면 재빌드 · 재배포

**메시가 대신 하는 것**

- 프록시가 mTLS를 종단
- 트래픽 정책을 선언으로 기술
- 언어 무관 — 프록시는 바깥에 있다
- 정책 변경 = CRD 수정

Istio는 이 아이디어의 **구현체 중 하나**다. Linkerd · Consul · Cilium Service Mesh도 있다.

# Istio는 k8s Add-On이다

CRD — networking · security · telemetry · extensions.istio.io 4개 API 그룹

Webhook — Mutating(사이드카 주입) · Validating(설정 검증)

istio-system 네임스페이스 — istiod Deployment

전제 조건 — 이미 동작하는 CNI

# istiod — 넷이 하나로

Pilot · Galley · Citadel · injector
각각 별도 배포 · 버전 · RBAC

→

istiod
단일 바이너리 (Istio 1.5, 2020-03)

| 컴포넌트 | 하던 일 |
|---|---|
| Pilot | 서비스 목록을 모아 **프록시 설정으로 바꿔 배포**. 지금 istiod의 본체 |
| Galley | 설정 **검증** + MCP로 배포. **검증만 살아남았다** (MCP는 1.5에서 기본 off → 소멸) |
| Citadel | **CA** — 워크로드 인증서 발급·갱신. 지금은 **끌 수도 있다**(외부 CA·SPIRE) |
| injector | 파드 생성을 가로채 **사이드카 컨테이너를 끼워 넣음** |

**이 네 이름은 모두 역사 용어다.** 현행 istio.io 아키텍처 문서에는 등장하지 않고 Envoy · Istiod 두 상자로만 설명한다(코드에 이미지명 `pilot`, `CA_PROVIDER=Citadel` 같은 잔재만 남음).

**용어** — "컨트롤플레인"은 **역할**이고, 쿠버네티스 오브젝트로는 `istio-system` 의 **평범한** Deployment다(차트 기본값에 `nodeSelector` · `tolerations` · `affinity` 가 전부 비어 있다). **쿠버네티스 컨트롤플레인**(kube-apiserver 등)과는 층이 다르다.

# istiod의 포트 네 개

| 포트 | 이름 | 용도 |
|---|---|---|
| 15010 | `grpc-xds` | XDS + CA — **평문**. 안전한 망에서만 |
| 15012 | `tls-xds` | XDS + CA — TLS/mTLS. **프로덕션 권장** |
| 15014 | `http-monitoring` | Prometheus 메트릭 |
| 15017 | `https-webhooks` | 주입 · 검증 webhook (Service 443 → 15017) |

# istiod가 죽으면

| 대상 | 결과 |
| --- | --- |
| **신규 파드** | 주입 webhook이 `failurePolicy: Fail` → 주입 대상 파드 생성이 **거부** |
| 기존 트래픽 | 프록시가 마지막 설정으로 계속 동작 (구조상 도출 — 1차 문구 미확인) |
| 실질 데드라인 | 인증서 갱신 중단. ztunnel 기본 TTL **24시간** |
| 완화 | `autoscaleMin: 2` + podAntiAffinity (노드 필수 · 존 선호) |

# 04 Istio Ambient mode

ztunnel · waypoint



# 사이드카에서 Ambient로 — 무엇이 바뀌나

**사이드카**

- 파드마다 Envoy 하나
- 유저스페이스 프록시 통과 양쪽 2회
- 풀 L7 처리
- 프록시를 바꾸려면 앱 파드 재시작

**Ambient**

- 노드마다 ztunnel 하나
- 유저스페이스 프록시 통과 양쪽 2회 — 동일
- L4만 처리
- 앱과 따로 교체

줄어든 것은 **프록시 개수**(파드당 → 노드당)와 **하는 일**(풀 L7 → L4)이다. 같은 노드 파드끼리도 로컬 ztunnel을 지난다.

홉(hop) = 패킷이 중간 지점을 한 번 지나가는 것. 여기서는 유저스페이스 프록시를 한 번 통과하는 걸 센다. 통과할 때마다 커널에서 프로세스로 올라갔다 다시 내려오므로 그만큼 지연이 붙는다.

# 구성요소는 세 개




`istio-cni-node` **는 이름과 달리 파드에 IP를 주지 않는다.** 하는 일이 두 개다.

① 기존 CNI 체인 **끝에 자기 바이너리를 얹는다** — 파드가 생길 때 호출되도록

② 상주 에이전트로 남아 파드 netns에 **리디렉션 규칙**을 넣고, ztunnel에 등록을 알린다

# ztunnel — Rust로 쓴 L4 전용 프록시

| 항목 | 내용 |
|---|---|
| 구현 | **Rust**. Envoy가 아니다 |
| 하는 일 | **L4까지** — mTLS 종단, 신원 기반 허용·차단 |
| 안 하는 일 | HTTP 라우팅 · 리트라이 · 타임아웃 → **waypoint 몫** |
| 터널 | **HBONE**, 포트 15008 |
| TLS | **1.3 고정** — 사이퍼는 `AES_128_GCM_SHA256` · `AES_256_GCM_SHA384` 둘뿐 |

**HTTP를 파싱할 일이 없다.** 그래서 ztunnel은 **노드 공유로 설계할 수 있었다.** 그 대가는 다음 장 — 키와 격리 경계가 노드 단위가 된다.

HBONE = 앱 트래픽을 HTTP/2 의 `CONNECT` 로 감싸 mTLS 터널 안에 넣는 방식. 겉에서 보면 15008 포트로 가는 TLS 스트림 하나로만 보인다.
**사이퍼수트** = 데이터를 무슨 알고리즘으로 암호화할지의 조합. TLS 1.3에서는 **AEAD + 해시** 둘만 남았고(키 교환·서명은 따로 협상해서 표준 조합이 5개로 줄었다), ztunnel은 그중 둘만 켠다 — ChaCha20은 없다.

# 인증서는 누가 갖고 있나

**그렇다 — 프록시가 앱 대신 갖는다.** 사이드카 Envoy가 하던 그 일을 ztunnel이 한다. 달라진 건 **보관 위치**다.

**사이드카 Envoy**

- 파드마다 프록시 하나
- 그 파드 신원의 인증서 하나
- 키는 프록시 프로세스 메모리
- 털리면 그 파드 하나

**ztunnel**

- 노드마다 프록시 하나
- 그 노드 모든 신원의 인증서
- 키는 프록시 프로세스 메모리 - 동일
- 털리면 그 노드 전부

**앱 컨테이너가 아무것도 갖지 않는 건 양쪽 다 같다.** 앱은 평문을 내보내고 자기 트래픽이 암호화되는 줄도 모른다.

# 그 인증서를 언제 받나 — 파드가 뜰 때

1. 주 CNI 가 veth 와 파드 IP 를 만든다
2. istio-cni 가 파드 안에 리디렉션 규칙을 넣는다
3. ztunnel 에 이 파드를 등록한다
4. 앱 컨테이너가 시작된다

**네 단계가 다 끝나야 앱이 시작된다.** 그래서 앱의 **첫 패킷부터** 규칙이 걸려 있다. 인증서는 등록 시점에 ztunnel이 istiod에서 받아 둔다.

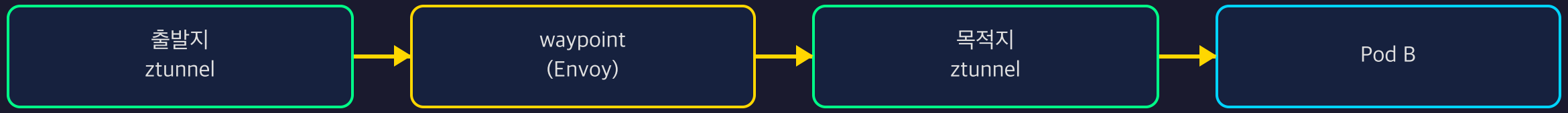
뒤집으면 - `istio-cni-node` 가 죽으면 **새 파드가 못 뜬다**( `ContainerCreating` 정지). 이미 떠 있는 파드는 영향 없다.

# istiod → ztunnel: 전송은 xDS, 타입은 전용




공식 표현은 "a simplified set of resources are used in the **xDS APIs**" — xDS를 안 쓰는 것이 아니라 **Envoy 타입 대신 Istio 전용 타입 2개**를 쓴다. 인증서도 같은 연결로 받는다.

# waypoint — L7이 필요할 때만




**누가 waypoint로 보내나** — 파드 안 방화벽 규칙이 아니라 **출발지 ztunnel**이 정한다. 목적지에 waypoint가 붙어 있다는 걸 알고 그쪽으로 먼저 보낸다. 그래서 waypoint를 붙이거나 떼도 파드 안 설정은 건드리지 않는다.

**비용 두 가지** — 프록시를 지나는 횟수가 2회에서 **3회**로 늘고, waypoint는 별도 Deployment라 **다른 노드에 떠 있을 수 있어** 그 노드까지 다녀오는 왕복이 추가된다.

# 05 Istio Ambient mode vs Cilium

① 실제로 뜨는 리소스 · ② L4·L7 · ③ L3

# ① 실제로 뜨는 리소스 - Istio Ambient

| 오브젝트 | 종류 | 개수 |
|---|---|---|
| `istiod` | Deployment | 1 (HPA 1~5) |
| `istio-cni-node` | DaemonSet | 노드 수 |
| `ztunnel` | DaemonSet | 노드 수 |
| `waypoint` | Deployment | **`Gateway`** 리소스마다 1 (아래 참조) |

**waypoint는 DaemonSet이 아니다.** `Gateway` 하나당 Deployment 하나 + Service·HPA·PDB. istiod가 `Gateway`를 보고 자동으로 만든다.

붙이는 **단위는 선택**이다 — 네임스페이스에 공용 하나로 둘 수도, 서비스·서비스어카운트마다 따로 둘 수도 있다.

여기서 Gateway는 **쿠버네티스 Gateway API**의 표준 리소스( `gateway.networking.k8s.io/v1` )다. `gatewayClassName: istio-waypoint` 로 하나 만들면 istiod가 그것을 보고 Deployment를 찍어낸다( `istioctl waypoint apply` 가 대신 만들어 준다).
**이름이 겹친다** — Istio 고유 API인 `networking.istio.io/v1 Gateway` (ingress·egress 게이트웨이 설정, **지금도 지원되는 안정 API**)와는 **별개 리소스**다. 그쪽은 메시 경계에 놓이고 waypoint는 **메시 안쪽**에 놓인다.

# ① 실제로 뜨는 리소스 - Cilium

**기본 설치** - 이 셋이 전부다

| 오브젝트 | 종류 | 개수 |
|---|---|---|
| `cilium` (agent) | DaemonSet | 노드 수 |
| `cilium-operator` | Deployment | 2 |
| `cilium-envoy` | DaemonSet | 노드 수 - **1.16부터 신규 설치 기본값** |

**선택 (opt-in)** - 켜야 뜬다

| 오브젝트 | 종류 | 조건 |
|---|---|---|
| `ztunnel-cilium` | DaemonSet | `encryption.type: ztunnel` - 1.19 Beta |

`cilium-envoy`는 예전엔 에이전트 파드 안의 프로세스였다. 지금은 별도 DaemonSet이 기본이고 **L7을 쓰든 안 쓰든 노드마다 하나 뜬다.**

# ① 규모를 넣어 보면 - 노드 50

| | Istio Ambient | Cilium |
|---|---|---|
| 노드 단위 | cni 50 + ztunnel 50 = **100** | agent 50 + envoy 50 = **100** |
| 그 외 | istiod 1~5 | operator 2 |
| **기본 합계** | **약 101~105** | **약 102** |
| 선택 추가 | waypoint - `Gateway` 마다 1 | ztunnel-cilium 50 (**Beta**) |

**기본은 거의 같다.** 갈리는 건 아래 줄이다 - Ambient는 **L7을 켤 때** `Gateway` 수만큼, Cilium은 **워크로드 mTLS를 켤 때** 노드 수만큼 늘어난다.

Ambient 쪽 숫자에는 **주 CNI가 빠져 있다** - Cilium이든 Calico든 무언가는 깔아야 하고 그것도 DaemonSet이다. 지연·처리량은 여기서 말하지 않는다: 공개된 head-to-head 수치가 없고, Istio 2025 벤치마크의 Cilium은 `v1.16.6` 으로 ztunnel 도입(1.19) 이전이다.

# ② 같은 L4·L7, 다른 뜻

**Cilium (ztunnel 없이)**

- 정책 판정 = 숫자 CiliumIdentity (라벨에서 파생)
- 커널 eBPF 에서 판정
- cilium-envoy 도 숫자 ID 로 매칭
- 인증서는 k8s Secret 에서 SDS 로

**Istio Ambient**

- 정책 판정 = SPIFFE 신원 (ns/sa)
- 유저스페이스 ztunnel 에서 판정
- IP 기반 (ipBlocks) 도 쓸 수 있다
- L7 은 waypoint - 붙인 범위만큼

**L7** - 차이는 성능이 아니라 **격리 범위**다. 노드 Envoy는 그 노드의 L7이 한곳에 모이고, waypoint는 붙인 범위만큼 나뉜다.

예외 하나 - **mutual authentication**은 SPIRE 기반이다. 다만 **1.19부터 기본 꺼짐 · 1.20 deprecated**(1.21 제거 예정)이고, 핸드셰이크는 Envoy가 아니라 **cilium-agent**가 노드 간 4250으로 한다. 신원도 `spiffe://.../identity/<숫자>` - 숫자 identity를 URI로 감싼 것이지 `ns/sa` 워크로드 신원이 아니다.

## ③ Cilium에만 있는 것 - 주소와 경로

Ambient는 **주 CNI 뒤에 붙는 플러그인**이라 아래 셋은 **책임 범위 밖**이다.

**IPAM** - 파드에 IP를 나눠주는 규칙. AWS ENI 모드면 파드가 **VPC IP를 직접** 받는다.
VPC 내부 목적지로 갈 때 오버레이 캡슐화도 노드 SNAT도 없다 (인터넷은 NAT GW를 거친다)

**BGP** - 파드 대역과 Service IP를 **외부 라우터에 광고**한다. 사내 장비가 그 경로를 배우면 클러스터 밖에서 파드 IP로 바로 붙는다.
클러스터 **안** 도달성을 만드는 게 아니라 **이미 있는 경로를 알리는** 기능이다

**LB IPAM** - 클라우드 LB가 없는 환경에서 `LoadBalancer` Service에 IP를 준다. 그 IP를 **알리는 건 BGP·L2 몫**이다.

## ③ Cilium에만 있는 것 - 출발지·노드·클러스터

**Egress Gateway** - 밖으로 나가는 파드 트래픽을 **지정한 노드**로 보내 **고정 IP로 SNAT**한다.
외부에서 방화벽에 출발지 IP 등록을 요구할 때 - 없으면 노드 IP를 전부 등록해야 한다
Istio의 동명 기능과 다르다 - 그쪽은 나가는 트래픽을 **모아서 정책·관측**을 걸 뿐 출발지 IP를 고정하지 않는다

**Host Firewall** - 정책 대상이 파드가 아니라 **노드 자신**이다. SSH·kubelet처럼 노드가 직접 여는 포트를 같은 문법으로 통제한다.

**ClusterMesh** - 클러스터 여러 개의 **파드망을 연결**한다. 같은 이름의 Service를 경계 너머로 묶는다.
ztunnel 암호화와 **동시 사용 불가** - 켜면 validation error